

INF1161

Réseaux mobiles et communications sans fil

UNIVERSITÉ TÉLUQ

Devoir 2

Simulation du protocole CSMA/CA à événements discrets

Date de remise : 22-05-2026

Présenté à : Es-Said Sabir

Étudiante : Melissa Moya

Numéro d'étudiant : 24332062

Table des matières

1. Introduction.....	3
2. Méthodologie	4
3. Architecture générale du simulateur.....	6
3.1 Moteur de simulation à événements discrets.....	7
3.2 Modèle des stations.....	7
3.3 Implémentation du protocole CSMA/CA.....	8
3.4 Calcul des métriques de performance	11
4. Choix techniques	12
5. Résultats expérimentaux	12
5.1 Performance sous charge modérée ($\lambda = 20$ paquets/s par station)	12
5.2 Performance sous forte charge ($\lambda = 200$ paquets/s par station).....	14
5.3 Impact de la taille minimale de la fenêtre de contention (W_{min})	15
5.4 Impact du nombre maximal de retransmissions (K_{max}).....	16
5.5 Impact du mécanisme RTS/CTS	17
6. Analyse et interprétation des résultats.....	18
7. Conclusion.....	19
Bibliographie	19
Annexe	20
Annexe A –Présentation du dépôt GitHub.....	20
Annexe B - Code source du simulateur intégralement commenté	21

1. Introduction

Les réseaux sans fil reposent sur des mécanismes efficaces de partage du médium afin de permettre à plusieurs stations de communiquer sur un même canal. Parmi ces mécanismes, le protocole CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) joue un rôle central dans les réseaux IEEE 802.11 en assurant une gestion équitable de l'accès au canal. Il constitue la base de la fonction DCF (Distributed Coordination Function), principale méthode d'accès au médium définie par ce standard.

Ce projet propose la conception et la mise en œuvre d'un simulateur à événements discrets du protocole CSMA/CA, visant à modéliser le fonctionnement de la couche MAC dans un environnement sans fil. L'écoute du canal est réalisée de manière abstraite en supposant que le simulateur dispose d'une vue globale des états de toutes les stations, conformément à l'hypothèse formulée dans le sujet. Le canal est partagé selon le mécanisme d'attente binaire exponentielle défini par IEEE 802.11, et les collisions ne sont détectées qu'après la tentative de transmission.

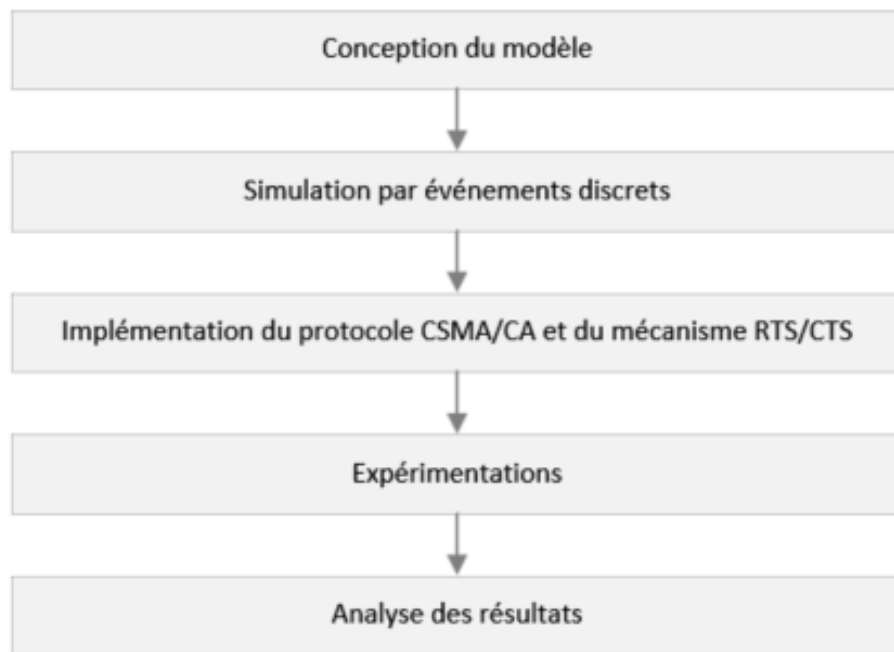
Les arrivées de paquets sont modélisées par des intervalles exponentiels formant un processus de renouvellement, soit qu'une station ne génère pas de nouveau paquet tant que le paquet courant n'a pas été transmis avec succès ou abandonné. Le comportement de chaque station est contrôlé par les paramètres W_{min} , W_{max} et K_{max} , ainsi que par les temporisations DIFS, SIFS et slot time, conformément au standard IEEE 802.11b. Une extension intégrant le mécanisme RTS/CTS et le vecteur d'allocation réseau (NAV) est également étudiée.

Les performances du protocole sont évaluées à l'aide de trois métriques principales, soit le débit moyen, le taux de collision et le délai moyen de transmission, en faisant varier le nombre de stations, la fenêtre de contention et le nombre maximal de retransmissions. La suite du rapport présente la méthodologie, l'architecture du simulateur, les choix techniques, puis les résultats expérimentaux et leur analyse.

2. Méthodologie

Le développement du simulateur repose sur une décomposition en cinq étapes clairement définies, comme illustré à la Figure 1.

Figure 1 – Étapes de conception du simulateur CSMA/CA



Dans un premier temps, un modèle à événements discrets a été retenu afin de représenter l'évolution du système dans le temps. Cette approche traite uniquement les événements significatifs, soit les arrivées de paquets, les tentatives de transmission, les mises à jour du backoff¹ et les fins de transmission, en les planifiant selon le slot time² du protocole.

Dans un second temps, un modèle de station a été défini pour représenter le comportement individuel de chaque entité du réseau. Chaque station maintient un état interne comprenant le paquet en cours de traitement, le compteur de backoff et le nombre de tentatives de retransmission. L'écoute du canal est modélisée de manière abstraite, le simulateur

¹**Backoff** (ou temporisation aléatoire) : délai d'attente tiré aléatoirement par chaque station avant de tenter une transmission, afin de réduire les risques de collision.

² **Slot time** (temps de slot) : unité de temps discrète du protocole IEEE 802.11, correspondant à la durée minimale d'un intervalle de contention ($\approx 20 \mu s$ en mode 802.11b).

disposant d'une vue globale des états de toutes les stations conformément à l'hypothèse du sujet. Chaque station ne traite qu'un seul paquet à la fois.

Dans un troisième temps, la logique du protocole CSMA/CA a été implémentée, incluant le mécanisme d'attente binaire exponentielle, la gestion des collisions et les retransmissions. Une extension intégrant le mécanisme RTS/CTS et la réservation du médium par le vecteur d'allocation réseau (NAV) a également été ajoutée, dont le fonctionnement est détaillé à la section 3.3.

Par la suite, des expérimentations ont été menées en faisant varier le nombre de stations, le taux d'arrivée des paquets et les paramètres du protocole, soit W_{min} , W_{max} , K_{max} et les temporisations DIFS, SIFS et slot time. Chaque configuration est simulée plusieurs fois et les résultats sont moyennés afin de réduire l'impact des fluctuations aléatoires.

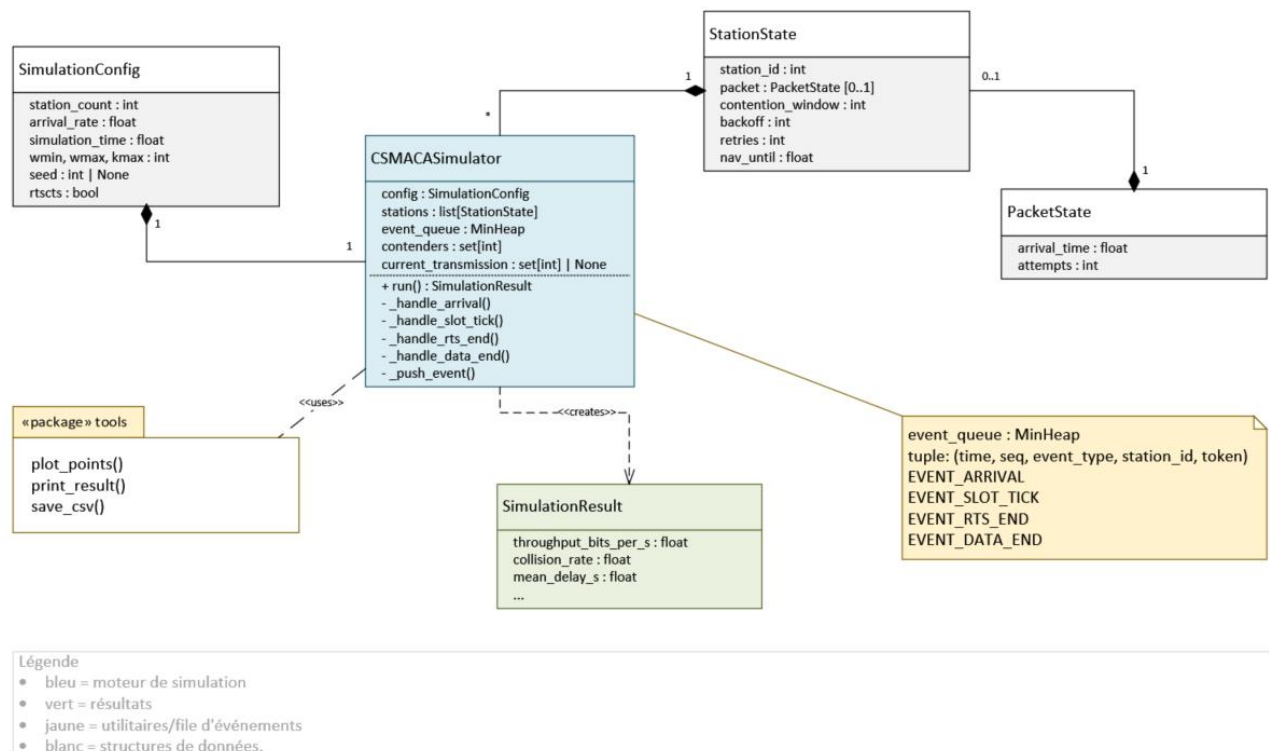
Enfin, les performances du système sont évaluées à l'aide de trois métriques principales, soit le débit moyen, le taux de collision et le délai moyen de transmission, permettant d'analyser le comportement du protocole selon les conditions de charge du réseau.

L'ensemble de ces étapes a guidé les décisions de conception du simulateur, dont l'architecture détaillée est présentée à la section suivante.

3. Architecture générale du simulateur

Le simulateur repose sur une architecture modulaire articulée autour de trois composants principaux, soit un moteur à événements discrets, un modèle de stations et un module d'implémentation du protocole et de calcul des métriques. La Figure 2 présente le diagramme de classes UML illustrant les interactions entre ces composants.

Figure 2 — Diagramme de classes UML du simulateur CSMA/CA



La classe centrale CSMACASimulator orchestre l'ensemble de la simulation. Elle maintient la liste des stations (StationState), gère la file d'événements (MinHeap) et produit un objet SimulationResult à l'issue de l'exécution. Chaque station encapsule son état courant, notamment le compteur de backoff, la fenêtre de contention, le nombre de tentatives et le vecteur NAV. Lorsqu'une station dispose d'un paquet à transmettre, celui-ci est représenté par un objet PacketState contenant l'heure d'arrivée et le nombre de tentatives effectuées. Enfin, le module tools regroupe les fonctions de visualisation et d'exportation des résultats. Les sections suivantes détaillent chacune de ces éléments.

3.1 Moteur de simulation à événements discrets

Le cœur du simulateur est un moteur à événements discrets dans lequel le temps progresse de manière discontinue en fonction des événements significatifs, soit les arrivées de paquets, les incréments de slot, les fins de transmission de données et, en mode RTS/CTS, les fins d'émission de trames RTS.

Les événements sont gérés à l'aide d'une file de priorité à tas minimal implémentée avec la bibliothèque `heapq`³ de Python, garantissant leur traitement dans l'ordre chronologique. Chaque événement est associé à un numéro de séquence monotone permettant de départager de manière déterministe les événements partageant le même instant, ce qui assure la reproductibilité des simulations. Cette approche évite une simulation à pas de temps constant, réduisant ainsi le nombre d'opérations aux seuls instants pertinents.

Un mécanisme de jeton permet d'invalidier les événements d'incrément de slot devenus obsolètes sans les retirer physiquement de la file, simplifiant la gestion des transitions d'état du canal. Le découplage entre la gestion temporelle et la logique des stations facilite l'évolution du modèle et son adaptation à différents scénarios.

3.2 Modèle des stations

Le simulateur considère un ensemble de stations indépendantes partageant un même canal de communication. Chaque station est modélisée par un état interne comprenant le paquet actif en cours de traitement, le compteur de backoff courant, le nombre de tentatives de retransmission effectuées ainsi qu'un champ de réservation indiquant jusqu'à quel instant le médium est réservé par le vecteur d'allocation réseau (NAV) en mode RTS/CTS.

Conformément aux hypothèses du sujet, une station ne traite qu'un seul paquet à la fois. Aucune nouvelle arrivée n'est planifiée tant que le paquet courant n'a pas été transmis avec succès ou abandonné après dépassement de k_{max} tentatives. En cas de succès, la fenêtre

³ `heapq` : module de la bibliothèque standard Python implémentant un tas binaire min (*min-heap*), utilisé ici comme file d'événements triée par ordre chronologique.

de contention est réinitialisée à W_{min} et un nouveau tirage de backoff est effectué pour le paquet suivant.

Le comportement des stations est régi par les paramètres du standard IEEE 802.11b, notamment les temporisations DIFS, SIFS et slot time.

3.3 Implémentation du protocole CSMA/CA

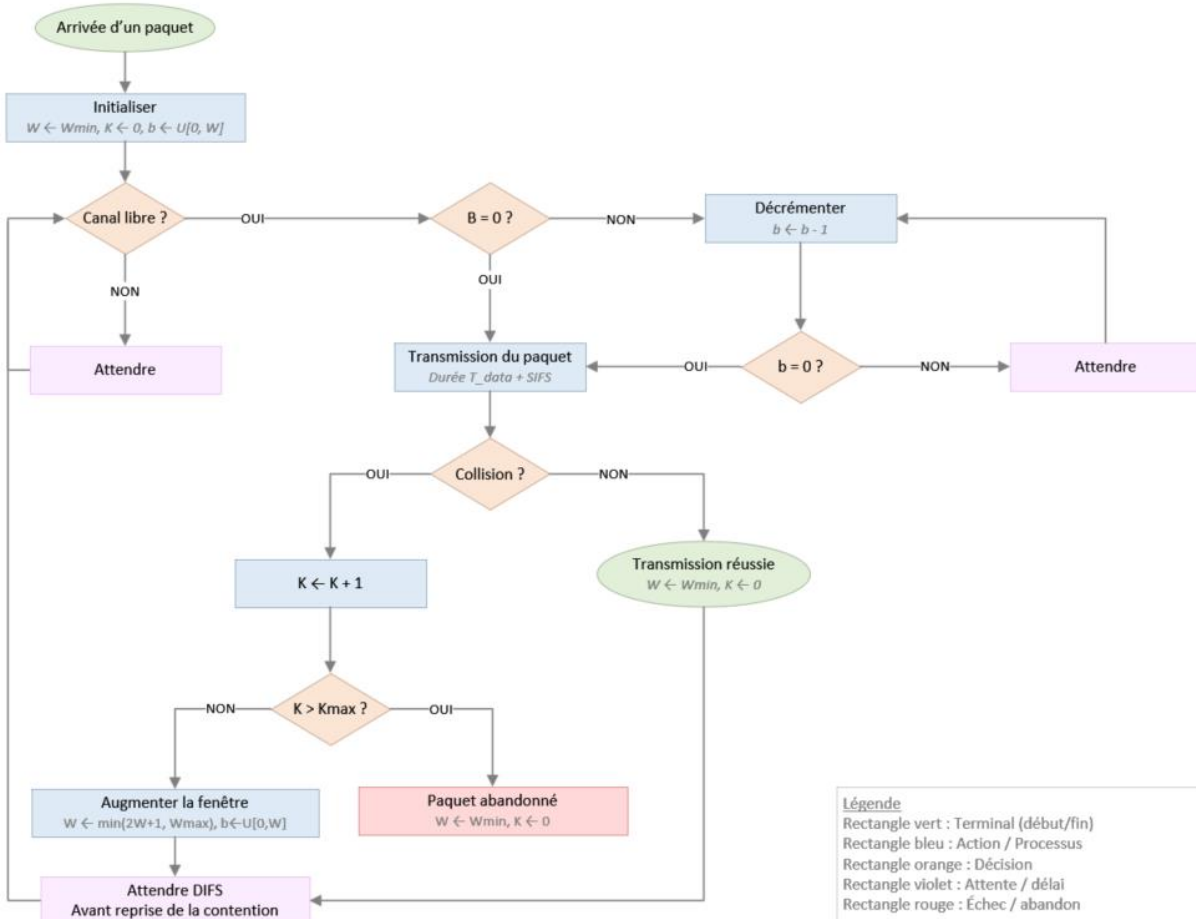
La logique du protocole CSMA/CA est intégrée directement dans le comportement des stations. Ce protocole repose sur quatre mécanismes fondamentaux définis par la fonction DCF, soit l'écoute du canal, l'algorithme d'attente binaire exponentielle, les espaces inter-trames (DIFS et SIFS) et les acquittements positifs.

Chaque station sélectionne un délai de backoff aléatoire b tiré uniformément dans l'intervalle $[0, W]$, puis surveille l'état du canal avant d'initier toute transmission. Le compteur de backoff est décrémenté d'un slot à chaque intervalle où le canal est libre et une transmission est initiée lorsque ce compteur atteint zéro. En cas de succès, un acquittement positif est implicitement modélisé après un délai SIFS. L'absence d'acquittement est interprétée comme un échec, déclenchant le mécanisme de retransmission.

Une collision se produit lorsque plusieurs stations atteignent simultanément un compteur de backoff nul. Chaque station impliquée incrémente son compteur de tentatives K , augmente sa fenêtre de contention selon la règle $W \leftarrow \min(2W + 1, W_{max})$, puis attend une période DIFS avant de reprendre la contention. Si K dépasse K_{max} , le paquet est abandonné et la station est réinitialisée à W_{min} .

La Figure 3 illustre ce processus sous forme d'organigramme, depuis l'arrivée d'un paquet jusqu'à sa transmission réussie ou son abandon. On y distingue trois chemins possibles : la transmission directe lorsque le canal est libre et le compteur de backoff atteint zéro, la retransmission avec doublement de la fenêtre de contention en cas de collision, et l'abandon du paquet lorsque K dépasse K_{max} .

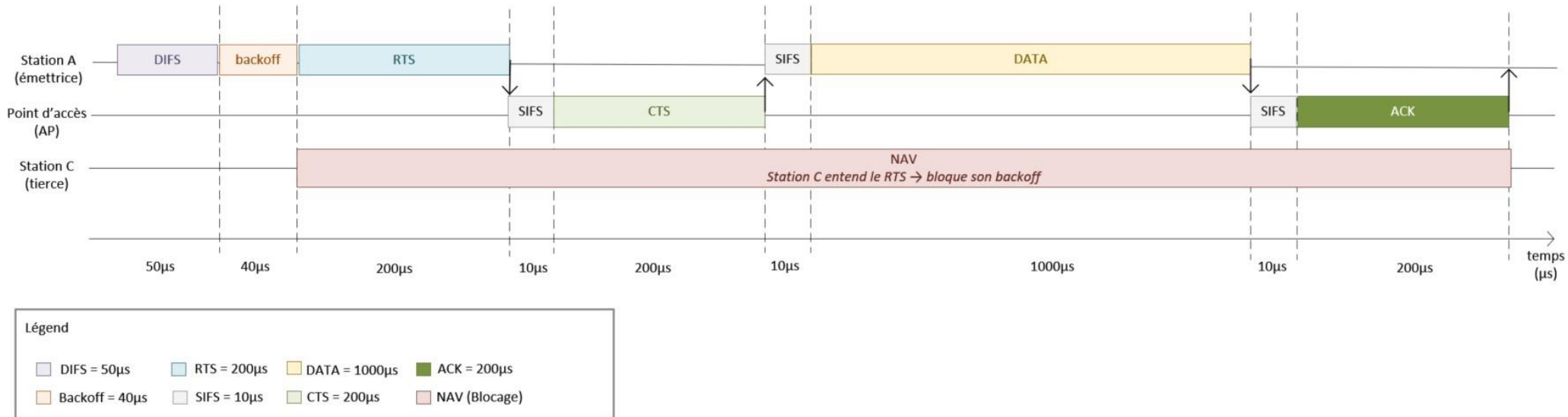
Figure 3 — Protocole CSMA/CA avec algorithme d'attente binaire exponentielle



Une extension intégrant le mécanisme RTS/CTS est également implémentée. Avant toute transmission, la station émettrice envoie une trame de demande d'émission (RTS). En l'absence de collision RTS, le point d'accès répond par une trame d'autorisation d'émission (CTS) et toutes les autres stations bloquent leur backoff via le vecteur d'allocation réseau (NAV) jusqu'à la fin prévue de la transmission. Ce mécanisme réduit les collisions sur les trames de données au prix d'un surcoût lié aux échanges de trames de contrôle supplémentaires.

Afin d'illustrer concrètement le déroulement temporel de cet échange, la Figure 4 présente le chronogramme complet d'une transmission avec mécanisme RTS/CTS entre la station émettrice, le point d'accès et une station tierce.

Figure 4 — Chronogramme du mécanisme RTS/CTS avec réservation NAV (IEEE 802.11)



On y observe que Station C, ayant capté la trame RTS, bloque son backoff pour toute la durée du vecteur NAV, empêchant ainsi toute collision durant la transmission des données. Chaque transition entre phases est séparée par un intervalle SIFS de 10 μs, garantissant un accès ordonné au canal. Le vecteur NAV est mis à jour par les stations tierces dès la réception de la trame RTS, qui contient dans son champ de durée le temps total nécessaire à l'échange CTS + DATA + ACK. Ainsi, même les stations qui n'entendent pas directement la trame DATA sont protégées contre les collisions, ce qui constitue l'avantage principal du mécanisme face au problème du terminal caché. Dans le simulateur, ce comportement est modélisé via l'attribut `nav_until` de chaque station, qui bloque le décrémentation du compteur de backoff tant que le temps courant de simulation n'a pas dépassé cette valeur. Il est à noter que les durées RTS et CTS sont toutes deux fixées à 200 μs, valeur arrondie qui abstrait les en-têtes PLCP de la couche physique conformément aux hypothèses de modélisation retenues.

3.4 Calcul des métriques de performance

Trois métriques principales sont calculées conformément aux définitions du sujet.

Le débit moyen correspond au nombre de bits transmis avec succès par unité de temps :

$$\text{Débit} = \frac{(\text{Nombre de paquets réussis} \times L)}{T_{sim}}$$

où $L = 12\,000$ bits et T_{sim} est la durée totale de la simulation.

Le taux moyen de collision représente la proportion de tentatives ayant abouti à une collision :

$$\text{Taux de collision} = \frac{(\text{Nombre de transmissions en collision})}{(\text{Nombre total de tentatives})}$$

Le délai moyen de transmission correspond au temps écoulé entre la génération d'un paquet et sa transmission réussie, moyenné sur l'ensemble des paquets transmis avec succès :

$$\text{Délai moyen} = \frac{(\sum(t_{succ} - t_{arr}))}{(\text{Nombre de paquets réussis})}$$

où T_{arr} est l'instant de génération et T_{succ} l'instant de fin de transmission réussie. Ce délai inclut les temps d'attente liés au backoff, aux retransmissions et aux périodes DIFS et SIFS.

En complément, le simulateur calcule l'utilisation du canal, soit la fraction du temps pendant laquelle le médium transporte des données utiles :

$$U = \frac{(\text{Nombre de paquets réussis} \times T_{data})}{T_{sim}}$$

où $T_{data} = 1$ ms correspond à la durée de transmission d'un paquet de 12 000 bits à 12 Mbps. Le simulateur comptabilise également les paquets abandonnés lorsque K_{max} tentatives ont été atteintes sans succès, permettant d'évaluer le taux de perte sous forte charge.

4. Choix techniques

Le simulateur est implémenté en Python, avec des structures de données définies à l'aide de classes de données (dataclasses) pour améliorer la lisibilité et réduire le code répétitif. Une compatibilité Python 3.8 et versions ultérieures est assurée en gérant dynamiquement le paramètre slots des classes de données, absent avant Python 3.10. Le code est organisé en modules distincts, séparant le moteur de simulation, le modèle des stations et les outils d'export et de visualisation.

La gestion des événements repose sur une file de priorité à tas minimal implémentée avec la bibliothèque `heapq`, garantissant un accès en temps logarithmique. Chaque événement est associé à un horodatage et à un numéro de séquence monotone permettant de départager de manière déterministe les événements simultanés. Un mécanisme de jeton invalide les événements d'incrément de slot obsolètes sans les retirer physiquement de la file.

La reproductibilité des résultats est assurée par un générateur aléatoire isolé (`random.Random`) initialisé avec une graine fixe. Les expériences sont répétées plusieurs fois et les résultats sont moyennés, avec calcul de l'écart-type, afin de réduire l'impact des fluctuations statistiques inhérentes au modèle.

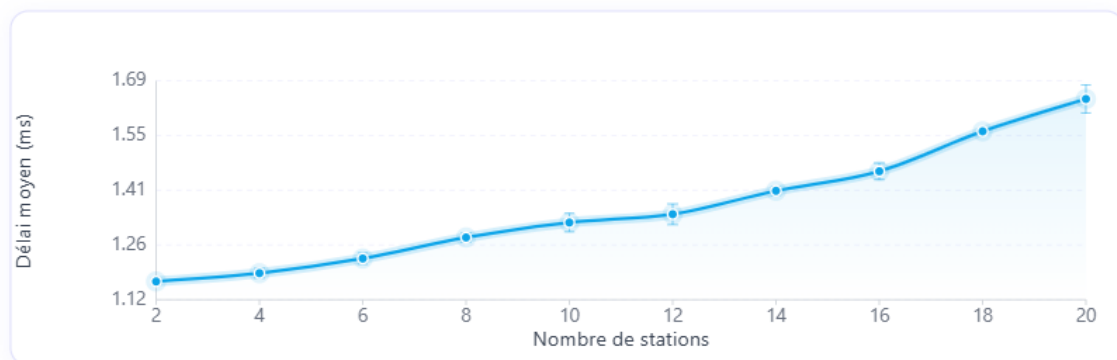
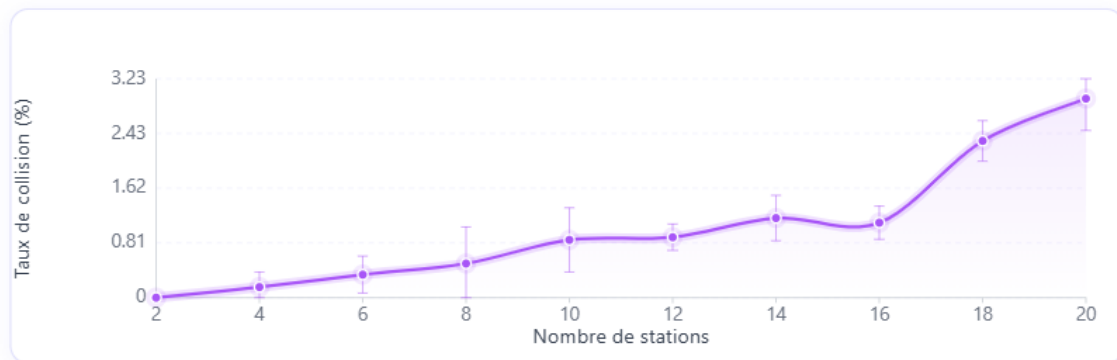
L'ensemble des paramètres du protocole, soit le nombre de stations, le taux d'arrivée, W_{min} , W_{max} et K_{max} , DIFS, SIFS, slot time et la durée de simulation, sont modifiables en ligne de commande, permettant d'explorer différents scénarios sans modifier le code source.

5. Résultats expérimentaux

Cinq scénarios ont été simulés en faisant varier le nombre de stations, la charge du réseau et les paramètres du protocole. Les résultats sont présentés sous forme de graphiques illustrant l'évolution du débit moyen, du taux de collision et du délai moyen de transmission, chaque configuration ayant été simulée plusieurs fois et les résultats moyennés.

5.1 Performance sous charge modérée ($\lambda = 20$ paquets/s par station)

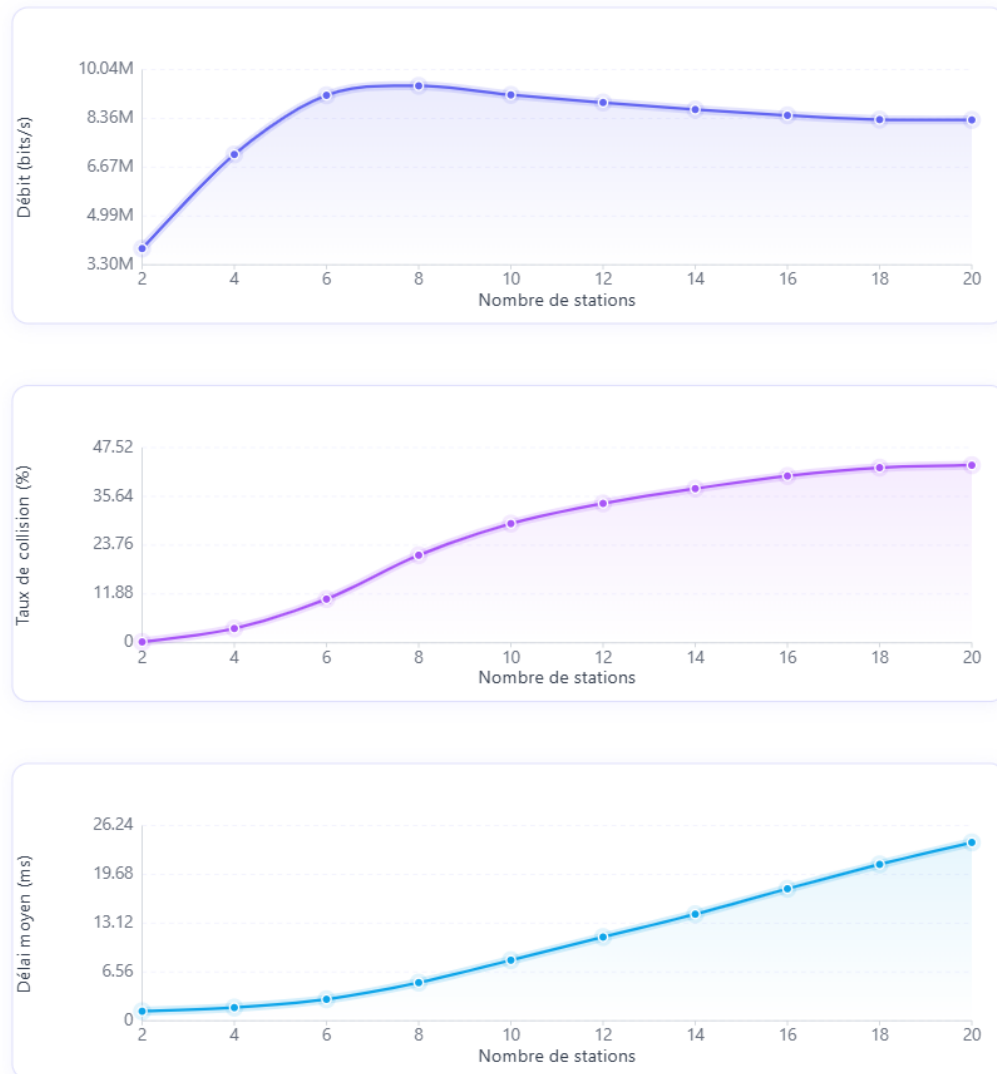
Figure 5 — Performances du protocole en fonction du nombre de stations (charge standard)



Le débit augmente de manière quasi linéaire avec le nombre de stations, atteignant environ 4,60 Mbps pour 20 stations, ce qui indique que le canal n'est jamais saturé dans ce régime. Le taux de collision demeure quasi nul (moins de 2,5 % pour 20 stations) et le délai moyen reste faible, passant d'environ 1,17 ms à 1,61 ms. Les barres d'erreur ($\pm 1\sigma$), visibles notamment sur le taux de collision à partir de $N=18$, indiquent une légère variabilité inter-répétitions liée à la nature stochastique du modèle, sans remettre en cause les tendances observées. Ces résultats confirment que le mécanisme d'attente binaire exponentielle gère efficacement la contention en charge modérée.

5.2 Performance sous forte charge ($\lambda = 200$ paquets/s par station)

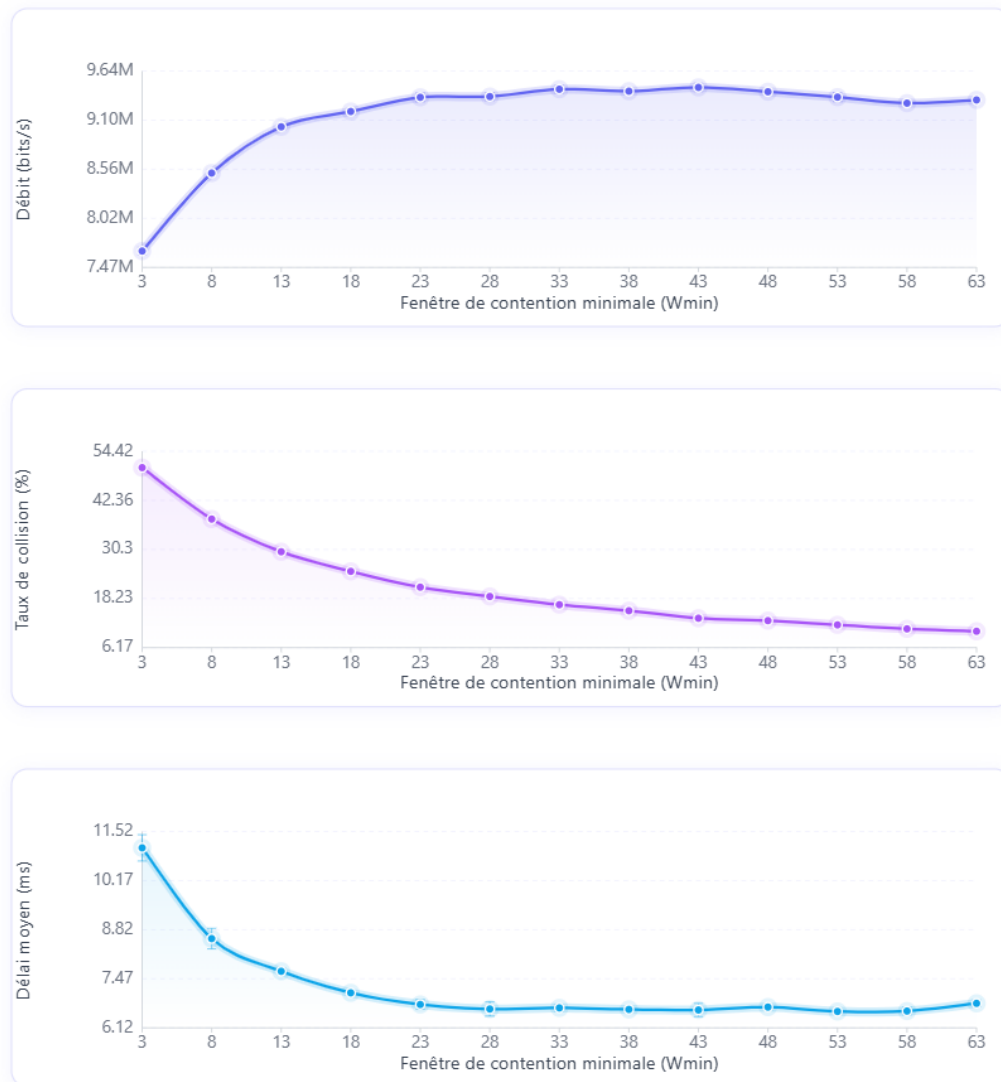
Figure 6 — Performances du protocole en situation de forte charge



Le débit croît rapidement jusqu'à $N=8$, atteint un pic de 9,42 Mbps, puis décline progressivement jusqu'à 8,23 Mbps à $N=20$, traduisant une saturation rapide du canal. Le délai moyen augmente fortement, passant de 1,29 ms à $N=2$ jusqu'à 24,19 ms à $N=20$, sous l'effet de l'algorithme d'attente binaire exponentielle qui allonge les intervalles d'attente à mesure que la contention s'intensifie. Le taux de collision croît de façon monotone, atteignant 43,8 % pour 20 stations. La dégradation des performances résulte donc de la combinaison de la contention croissante et des collisions, et non d'un seul de ces facteurs isolément.

5.3 Impact de la taille minimale de la fenêtre de contention (W_{min})

Figure 7 — Impact de W_{min} sur les performances du protocole



Pour $W_{min}=3$, le taux de collision atteint 50,3 % et le délai est maximal (10,84 ms), car les stations tirent des délais d'attente très courts et entrent fréquemment en conflit. En augmentant W_{min} , le débit croît et le taux de collision diminue de façon monotone. Le débit est maximisé autour de $W_{min}=33$ (9,48 Mbps) et le délai minimal autour de $W_{min}=38$ (6,49 ms). Au-delà, le débit redéccline légèrement car les stations attendent inutilement longtemps même lorsque le canal est disponible. Ces résultats confirment que la valeur

standard IEEE 802.11b de $W_{min}=15$ constitue un compromis conservateur, la zone optimale se situant entre 28 et 38 dans ce scénario.

5.4 Impact du nombre maximal de retransmissions (K_{max})

Figure 8 — Impact de K_{max} sur les performances du protocole

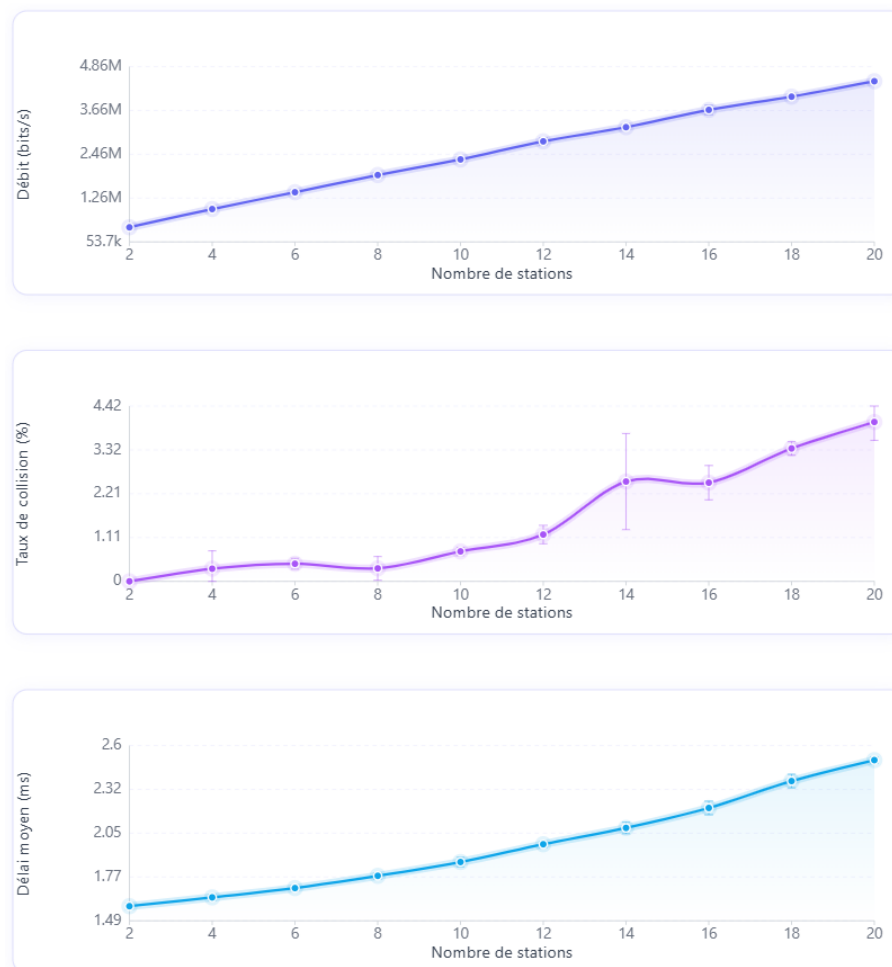


Pour $K_{max}=1$, le débit est minimal (7,62 Mbps) et le délai est le plus faible (8,78 ms), car les paquets sont abandonnés rapidement après un seul échec, libérant le canal mais au prix d'un taux d'abandon élevé. Le taux de collision est également maximal (51,3 %), le backoff n'ayant pas le temps de diverger entre les stations. En augmentant K_{max} de 1 à 6, le débit croît significativement et se stabilise autour de 8,65 Mbps, le taux de collision diminue vers 38 % et le délai se stabilise autour de 13-14 ms. Au-delà de $K_{max}=6$, les métriques n'évoluent plus de façon significative, car le protocole dispose de suffisamment de

tentatives pour que l'algorithme d'attente binaire exponentielle converge sans accumuler des attentes inutiles. Une valeur de K_{max} comprise entre 6 et 8 constitue un bon compromis dans ce scénario.

5.5 Impact du mécanisme RTS/CTS

Figure 9 — Performances du protocole avec mécanisme RTS/CTS



Le mécanisme RTS/CTS protège les trames de données contre les collisions via le vecteur d'allocation réseau (NAV), mais introduit un surcoût lié aux échanges de trames de contrôle supplémentaires. Le débit utile est limité à 4,60 Mbps pour 20 stations, inférieur au cas standard, et le délai augmente jusqu'à 2,56 ms. Des collisions subsistent au niveau des trames de demande d'émission elles-mêmes, atteignant 4,29 % pour 20 stations. Dans un environnement sans terminal caché, l'utilisation de RTS/CTS ne permet pas d'améliorer les

performances globales et introduit un compromis défavorable entre protection des données et surcoût protocolaire.

6. Analyse et interprétation des résultats

En régime de charge modérée, le mécanisme d'attente binaire exponentielle maintient un taux de collision quasi nul et un délai stable, confirmant l'efficacité du protocole lorsque la charge offerte reste inférieure à la capacité du canal.

En régime de forte charge, la saturation du canal se manifeste par un pic de débit à $N=8$ suivi d'un déclin, tandis que le délai croît de façon exponentielle sous l'effet de la règle $W \leftarrow \min(2W+1, W_{max})$. Ce comportement illustre la limite fondamentale du protocole CSMA/CA en milieu dense, où la contention et les collisions se renforcent mutuellement.

L'analyse paramétrique révèle deux zones optimales. Pour W_{min} , la zone optimale se situe entre 28 et 38, nettement au-dessus de la valeur standard IEEE 802.11b de 15, qui s'avère trop conservatrice en situation de forte charge. Pour K_{max} , une valeur entre 6 et 8 suffit à permettre la convergence du backoff sans accumulation d'attentes inutiles, au-delà de laquelle les gains deviennent négligeables.

Concernant le mécanisme RTS/CTS, son bénéfice reste limité dans un environnement sans terminal caché. Le surcoût protocolaire induit par les échanges de trames de contrôle réduit le débit utile et augmente le délai sans amélioration significative du taux de collision sur les trames de données, ce qui est cohérent avec les observations formulées dans les notes de cours concernant l'inefficacité de ce mécanisme en milieu à faible contention.

Ces résultats mettent en évidence les compromis inhérents au protocole CSMA/CA entre efficacité du canal, délai de transmission et complexité protocolaire, et soulignent l'importance du réglage des paramètres W_{min} et K_{max} selon les conditions de charge du réseau.

7. Conclusion

Ce projet a permis de concevoir et d'implémenter un simulateur à événements discrets du protocole CSMA/CA, modélisant la couche MAC d'un réseau IEEE 802.11 avec gestion du backoff exponentiel, des collisions, des retransmissions et du mécanisme RTS/CTS.

Les expérimentations menées confirment que le protocole gère efficacement la contention en charge modérée, mais se dégrade significativement en situation de saturation, où la contention et les collisions se renforcent mutuellement. L'analyse paramétrique montre que les valeurs par défaut de W_{min} et K_{max} définies par le standard IEEE 802.11b sont conservatrices et qu'une optimisation selon les conditions de charge permet d'améliorer sensiblement les performances. Le mécanisme RTS/CTS, s'il réduit les collisions sur les trames de données, introduit un surcoût protocolaire qui s'avère défavorable en l'absence de terminaux cachés.

Des extensions futures pourraient inclure la modélisation de terminaux cachés, l'intégration d'un canal physique bruité ou l'adaptation du simulateur à des topologies multi-cellules, afin d'évaluer le protocole dans des conditions plus proches de déploiements réels.

Bibliographie

Python Software Foundation. Python Documentation. Consulté en 2026. Disponible à : <https://docs.python.org>

Sabir, E. Notes de cours INF1161 – Simulation et protocoles réseaux, Modules 4 et 5. Université TÉLUQ, consulté en 2026.

IEEE. (2020). IEEE Std 802.11-2020: Wireless LAN Medium Access Control (MAC) and Physical Layer Specifications. Disponible à : <https://ieeexplore.ieee.org/document/9363693>

Annexe

Annexe A –Présentation du dépôt GitHub

Le code source du simulateur ainsi que les outils nécessaires à la reproduction des résultats sont disponibles dans un dépôt GitHub dédié.

Le dépôt contient les éléments suivants :

- Le programme principal du simulateur CSMA/CA (`csma_ca_sim.py`), implémentant le mécanisme d'accès au médium avec attente binaire exponentielle et, en option, le mécanisme RTS/CTS
- Un ensemble d'utilitaires (tools) regroupant les modules de génération de graphiques SVG (`plot.py`), d'affichage des résultats (`print_result.py`), d'export CSV (`save_csv.py`) et le script de reproduction des expériences (`run_experiments.py`)
- Les fichiers de données générés à partir des simulations, enregistrés dans le répertoire data sous forme de fichiers CSV
- Les graphiques produits automatiquement à partir des résultats des simulations, disponibles dans le répertoire figures
- Un diagramme illustrant le fonctionnement du protocole (répertoire assets)
- Un fichier de tests unitaires (`test_csma_ca_sim.py`), assurant la validation du comportement du simulateur et couvrant les principaux cas d'exécution
- Une documentation (`README.md`) décrivant l'installation, l'utilisation et les différentes options du simulateur
- Un fichier de dépendances (`requirements-dev.txt`) nécessaire à l'exécution des tests

Le dépôt est structuré de manière à assurer la reproductibilité complète des résultats présentés dans ce rapport. L'ensemble des expériences peut être reproduit automatiquement à l'aide de la commande suivante :

```
python tools/run_experiments.py
```

Ce script exécute successivement les différentes configurations expérimentales, génère les fichiers de données dans le répertoire data et produit les graphiques correspondants dans le répertoire figures.

Lien vers le dépôt : [moyamelissa/Mobile-Networks-and-Wireless-Communications](https://github.com/moyamelissa/Mobile-Networks-and-Wireless-Communications)

Annexe B - Code source du simulateur intégralement commenté

```
"""Simulateur à événements discrets du protocole CSMA/CA (IEEE 802.11, BEB,
RTS/CTS optionnel).

Usage :
    python csma_ca_sim.py --stations 8 --arrival-rate 20 --simulation-time 20
    python csma_ca_sim.py --sweep-stations 2 20 2 --runs 3 --output
    resultats.svg

Graphiques : tools/plot.py – Expériences : tools/run_experiments.py
"""
from __future__ import annotations

import argparse
import heapq
import math
import random
from dataclasses import dataclass as _dataclass

# `slots` n'existe que depuis Python 3.10 – retiré silencieusement si absent.
_DATACLASS_SUPPORTS_SLOTS = True
try:
    _dataclass(slots=True)(type("_X", (), {}))
except TypeError: # pragma: no cover
    _DATACLASS_SUPPORTS_SLOTS = False # pragma: no cover

def dataclass_compat(**kwargs):
    """Décorateur @dataclass compatible 3.8+ (retire `slots` si non
    supporté)."""
    if not _DATACLASS_SUPPORTS_SLOTS and "slots" in kwargs:
        kwargs = {k: v for k, v in kwargs.items() if k != "slots"}
    return _dataclass(**kwargs)

from pathlib import Path
from statistics import mean, pstdev
from typing import Optional

from tools.plot import plot_points # graphique SVG (tools/plot.py)
from tools.print_result import print_result # affichage console
(tools/print_result.py)
from tools.save_csv import save_csv # export CSV (tools/save_csv.py)

# -----
# Types d'événements du moteur de simulation
# -----
EVENT_ARRIVAL = "arrival" # Arrivée d'un paquet dans une station
EVENT_SLOT_TICK = "slot_tick" # Tick de slot de backoff
EVENT_RTS_END = "rts_end" # Fin d'émission RTS (mode RTS/CTS)
EVENT_DATA_END = "data_end" # Fin d'émission d'une trame de données

def next_slot_boundary(time_value: float, slot_time: float) -> float:
    """Retourne le plus petit multiple de slot_time >= time_value.
```

```

    Le décalage 1e-15 corrige les erreurs d'arrondi sur les flottants déjà
    alignés.
    Lève ValueError si slot_time <= 0.
    """
    if slot_time <= 0:
        raise ValueError("slot_time must be positive")
    scaled = (time_value - 1e-15) / slot_time
    slot_index = math.ceil(scaled)
    return max(0.0, slot_index * slot_time)

@dataclass_compat(slots=True)
class SimulationConfig:
    """Paramètres MAC et temporels de la simulation (défauts : IEEE
    802.11b)."""
    station_count: int = 8          # Nombre de stations en compétition pour le
    canal
    arrival_rate: float = 20.0      # Taux d'arrivée par station (paquets/s) –
    processus de renouvellement
    simulation_time: float = 20.0   # Horizon de simulation (secondes)
    packet_bits: int = 12000        # Taille d'un paquet (bits) – 12 000 bits =
    1 500 octets
    packet_duration: float = 0.001  # Durée de transmission d'un paquet
    (secondes)
    slot_time: float = 20e-6        # Durée d'un slot de backoff (secondes)
    difs: float = 50e-6            # DIFS : délai inter-trames distribué
    (secondes)
    sifs: float = 10e-6            # SIFS : délai inter-trames court
    (secondes)
    wmin: int = 15                  # Fenêtre de contention minimale W_min
    wmax: int = 1023                # Fenêtre de contention maximale W_max
    kmax: int = 15                  # Nombre maximal de tentatives avant
    abandon du paquet
    seed: Optional[int] = None      # Graine aléatoire (None = non
    déterministe)
    rtscs: bool = False            # Active le mécanisme RTS/CTS et le NAV si
    True
    rts_duration: float = 200e-6    # Durée d'une trame RTS (secondes)
    cts_duration: float = 200e-6    # Durée d'une trame CTS (secondes)

@dataclass_compat(slots=True)
class PacketState:
    """Trame active dans une station (créée à l'arrivée, détruite après
    succès ou abandon)."""
    arrival_time: float            # Instant de génération du paquet – sert à calculer
    le délai moyen
    attempts: int = 0              # Nombre de tentatives de transmission déjà
    effectuées pour ce paquet

@dataclass_compat(slots=True)
class StationState:
    """État MAC d'une station : une seule trame active (None = inactive),
    compteurs BEB et NAV."""
    station_id: int                # Identifiant unique de la station
    (index dans la liste)

```

```

    packet: Optional[PacketState] = None # Trame active (None = station
inactive)
    contention_window: int = 0           # Fenêtre de contention courante  $W \in$ 
[W_min, W_max]
    backoff: int = 0                    # Compteur de backoff courant  $b \in$ 
[0, W]
    retries: int = 0                    # Nombre de tentatives échouées pour
la trame courante (K)
    nav_until: float = 0.0              # Instant jusqu'auquel le médium est
réservé (NAV)

@dataclass_compat(slots=True)
class SimulationResult:
    """Métriques agrégées retournées par run_single_experiment() et
average_results()."""
    throughput_packets_per_s: float      # Débit en paquets transmis avec succès
par seconde
    throughput_bits_per_s: float          # Débit binaire utile (bits/s)
    channel_utilization: float            # Fraction du temps où le canal
transporte des données utiles
    offered_load_packets_per_s: float     # Charge offerte totale (paquets
générés / s)
    collision_rate: float                  # Proportion des tentatives ayant
abouti à une collision
    mean_delay_s: float                   # Délai moyen entre génération et
transmission réussie (s)
    generated_packets: int                 # Nombre total de paquets générés
durant la simulation
    successful_packets: int                # Nombre de paquets transmis avec
succès
    dropped_packets: int                   # Nombre de paquets abandonnés (K >
K_max)
    total_attempts: int                    # Nombre total de tentatives de
transmission
    collided_packets: int                  # Nombre de tentatives ayant abouti à
une collision

@dataclass_compat(slots=True)
class ExperimentPoint:
    """Point de courbe : valeur du paramètre balayé + métriques moyennes et
écarts-types ( $\pm\sigma$ )."""
    x_value: int                           # Valeur du paramètre balayé (ex.
nombre de stations)
    throughput_packets_per_s: float          # Débit moyen (paquets/s)
    throughput_bits_per_s: float             # Débit binaire moyen (bits/s)
    collision_rate: float                    # Taux de collision moyen
    mean_delay_s: float                     # Délai moyen de transmission
(secondes)
    throughput_bits_std: float = 0.0         # Écart-type du débit binaire entre
les répétitions (bits/s)
    collision_rate_std: float = 0.0          # Écart-type du taux de collision
entre les répétitions
    mean_delay_std: float = 0.0              # Écart-type du délai moyen entre les
répétitions (secondes)

```

```

class CSMACASimulator:
    """Moteur à événements discrets CSMA/CA. Instancier puis appeler
    run()."""

    def __init__(self, config: SimulationConfig):
        """Initialise le simulateur avec la configuration donnée."""
        self.config = config
        self.random = random.Random(config.seed) # RNG isolé – reproductible
via config.seed
        self.stations = [StationState(station_id=i) for i in
range(config.station_count)]

        # Min-heap : (temps, séquence, type_événement, station_id, jeton)
        # Séquence monotone pour ordre stable à temps égaux.
        self.event_queue: list[tuple[float, int, str, int, Optional[int]]] =
[]

        self.sequence = 0

        self.contenders: set[int] = set() # Stations en
phase de backoff
        self.current_transmission: Optional[set[int]] = None # En
transmission (None = libre ; len > 1 = collision)
        self.contention_open_time = 0.0 # Ouverture de
contention après DIFS
        self.scheduled_slot_tick_time: Optional[float] = None # Prochain
slot_tick prévu
        self.slot_tick_token = 0 # Jeton
d'invalidation des slot_ticks périmés

        # Compteurs cumulés – agrégés dans run()
        self.generated_packets = 0
        self.successful_packets = 0
        self.dropped_packets = 0
        self.total_attempts = 0
        self.collided_packets = 0
        self.successful_bits = 0
        self.delay_sum = 0.0

    def run(self) -> SimulationResult:
        """Planifie les premières arrivées, traite la file d'événements,
retourne les métriques."""
        # Première arrivée de chaque station (loi exponentielle)
        for station_id in range(self.config.station_count):
            first_arrival = self._sample_interarrival()
            if first_arrival <= self.config.simulation_time:
                self._push_event(first_arrival, EVENT_ARRIVAL, station_id)

        # Boucle principale : traite les événements par ordre chronologique
        while self.event_queue:
            time_value, _, event_type, station_id, token =
heapq.heappop(self.event_queue)

            if event_type == EVENT_SLOT_TICK:
                if token != self.slot_tick_token:
                    continue # pragma: no cover

```



```

        self.scheduled_slot_tick_time = None

    if event_type == EVENT_ARRIVAL:
        self._handle_arrival(time_value, station_id)
    elif event_type == EVENT_SLOT_TICK:
        self._handle_slot_tick(time_value)
    elif event_type == EVENT_RTS_END:
        self._handle_rts_end(time_value)
    elif event_type == EVENT_DATA_END:
        self._handle_data_end(time_value)
    else:
        raise RuntimeError(f"Unknown event type: {event_type}")

    # Métriques finales
    throughput_packets_per_s = self.successful_packets /
self.config.simulation_time if self.config.simulation_time > 0 else 0.0
    throughput_bits_per_s = self.successful_bits /
self.config.simulation_time if self.config.simulation_time > 0 else 0.0
    channel_utilization = (
        (self.successful_packets * self.config.packet_duration) /
self.config.simulation_time
        if self.config.simulation_time > 0 and
self.config.packet_duration > 0
        else 0.0
    )
    offered_load_packets_per_s = self.generated_packets /
self.config.simulation_time if self.config.simulation_time > 0 else 0.0
    collision_rate = self.collided_packets / self.total_attempts if
self.total_attempts > 0 else 0.0
    mean_delay_s = self.delay_sum / self.successful_packets if
self.successful_packets > 0 else 0.0

    return SimulationResult(
        throughput_packets_per_s=throughput_packets_per_s,
        throughput_bits_per_s=throughput_bits_per_s,
        channel_utilization=channel_utilization,
        offered_load_packets_per_s=offered_load_packets_per_s,
        collision_rate=collision_rate,
        mean_delay_s=mean_delay_s,
        generated_packets=self.generated_packets,
        successful_packets=self.successful_packets,
        dropped_packets=self.dropped_packets,
        total_attempts=self.total_attempts,
        collided_packets=self.collided_packets,
    )

    def _push_event(self, time_value: float, event_type: str, station_id:
int, token: Optional[int] = None) -> None:
        """Insère un événement dans la file (séquence monotone pour ordre
stable à temps égaux)."""
        self.sequence += 1
        heapq.heappush(self.event_queue, (time_value, self.sequence,
event_type, station_id, token))

    def _sample_interarrival(self) -> float:
        """Inter-arrivée ~ Exp( $\lambda$ ). Processus de renouvellement (non Poisson
pur).

```

```

    Retourne inf si arrival_rate <= 0.
    """
    if self.config.arrival_rate <= 0:
        return math.inf
    return self.random.expovariate(self.config.arrival_rate)

def _sample_backoff(self, contention_window: int) -> int:
    """Tire b ~ U[0, contention_window] (règle BEB IEEE 802.11)."""
    return self.random.randint(0, contention_window)

def _prime_station_for_contention(self, station_id: int, current_time:
float) -> None:
    """Initialise W=W_min, K=0, tire b et enregistre la station en
contention."""
    station = self.stations[station_id]
    if station.packet is None:
        return # Pas de paquet actif : rien à faire

    # W=W_min, K=0 pour le premier essai (IEEE 802.11)
    station.contention_window = self.config.wmin
    station.retries = 0
    station.backoff = self._sample_backoff(station.contention_window)
    self.contenders.add(station_id)

    # Déclenche le slot_tick si le canal est libre
    if self.current_transmission is None:
        self._schedule_slot_tick_if_needed(current_time)

def _schedule_next_arrival(self, station_id: int, base_time: float) ->
None:
    """Planifie la prochaine arrivée après résolution du paquet courant
(dans l'horizon)."""
    next_arrival = base_time + self._sample_interarrival()
    if next_arrival <= self.config.simulation_time:
        self._push_event(next_arrival, EVENT_ARRIVAL, station_id)

def _schedule_slot_tick_if_needed(self, current_time: float) -> None:
    """Planifie le prochain slot_tick si le canal est libre et des
stations attendent.
    Le mécanisme de jeton invalide les slot_ticks périmés sans les
retirer de la file.
    """
    if self.current_transmission is not None or not self.contenders:
        return # Canal occupé ou aucune station en attente

    # Respecte la période DIFS post-transmission (contention_open_time)
    candidate = next_slot_boundary(max(current_time,
self.contention_open_time), self.config.slot_time)
    if self.scheduled_slot_tick_time is not None and
self.scheduled_slot_tick_time <= candidate:
        return # Slot_tick antérieur déjà planifié

    # Invalide le slot_tick précédent via le jeton
    self.slot_tick_token += 1
    self.scheduled_slot_tick_time = candidate
    self._push_event(candidate, EVENT_SLOT_TICK, -1,
self.slot_tick_token)

```

```

    def _start_transmission(self, current_time: float, station_ids:
list[int]) -> None:
    """Démarré une transmission DATA. Collision si len > 1 (sans SIFS) ;
succès avec SIFS. Planifie DATA_END."""
    if not station_ids:
        return

    self.current_transmission = set(station_ids)
    for station_id in station_ids:
        self contenders.discard(station_id) # Quitte la phase de
contention
        self.stations[station_id].packet.attempts += 1 # type:
ignore[union-attr]

    self.total_attempts += len(station_ids)
    if len(station_ids) > 1:
        # Collision : trame corrompue, pas de SIFS
        release_time = current_time + self.config.packet_duration
    else:
        # Succès : SIFS avant ACK implicite
        release_time = current_time + self.config.packet_duration +
self.config.sifs

    # Canal occupé - invalide les slot_ticks en attente
    self.slot_tick_token += 1
    self.scheduled_slot_tick_time = None
    self._push_event(release_time, EVENT_DATA_END, -1)

    def _start_rts(self, current_time: float, station_ids: list[int]) ->
None:
    """Démarré une émission RTS. Planifie RTS_END ; collision/succès
résolus dans _handle_rts_end."""
    if not station_ids:
        return

    self.current_transmission = set(station_ids)
    for station_id in station_ids:
        self contenders.discard(station_id) # Quitte la phase de
contention (entre en RTS)
        self.stations[station_id].packet.attempts += 1 # type:
ignore[union-attr]

    self.total_attempts += len(station_ids)
    rts_end = current_time + self.config.rts_duration
    # Canal occupé durant l'émission RTS - invalide les slot_ticks
    self.slot_tick_token += 1
    self.scheduled_slot_tick_time = None
    self._push_event(rts_end, EVENT_RTS_END, -1)

    def _handle_arrival(self, current_time: float, station_id: int) -> None:
    """Crée le PacketState, incrémente le compteur et amorce la
contention (W, K, b)."""
    station = self.stations[station_id]
    if station.packet is not None:
        # Impossible avec la génération interne (paquet précédent non
résolu)

```

```

        return

    self.generated_packets += 1
    station.packet = PacketState(arrival_time=current_time)
    self._prime_station_for_contention(station_id, current_time)

    def _handle_slot_tick(self, current_time: float) -> None:
        """Décrémente les backoffs des stations actives (hors NAV) et
        déclenche la transmission si b=0."""
        if self.current_transmission is not None or not self.contenders:
            return # Canal occupé ou aucune station en contention

        # Stations hors NAV uniquement
        active = [s for s in self.contenders if self.stations[s].nav_until <=
current_time]

        # Étape 1 : b=0 déjà atteint – transmission immédiate
        ready_to_send = [station_id for station_id in active if
self.stations[station_id].backoff == 0]
        if ready_to_send:
            if self.config.rtscts:
                self._start_rts(current_time, ready_to_send)
            else:
                self._start_transmission(current_time, ready_to_send)
            return

        # Étape 2 : décrémentation
        for station_id in active:
            station = self.stations[station_id]
            if station.backoff > 0:
                station.backoff -= 1

        # Étape 3 : b=0 après décrémentation
        active_after = [station_id for station_id in active if
self.stations[station_id].backoff == 0]
        if active_after:
            if self.config.rtscts:
                self._start_rts(current_time, active_after)
            else:
                self._start_transmission(current_time, active_after)
            return

        # Étape 4 : aucune station prête – prochain slot
        self._schedule_slot_tick_if_needed(current_time +
self.config.slot_time)

    def _handle_rts_end(self, current_time: float) -> None:
        """Fin RTS : collision → BEB ou abandon ; succès → NAV sur les autres
        stations + planifie DATA_END."""
        if self.current_transmission is None:
            return

        if len(self.current_transmission) > 1:
            # Collision RTS
            affected_stations = list(self.current_transmission)
            self.collided_packets += len(affected_stations)
            self.current_transmission = None

```

```

        self.contention_open_time = current_time + self.config.difs #
Attente DIFS avant re-contention

        for station_id in affected_stations:
            station = self.stations[station_id]
            station.retries += 1
            if station.retries > self.config.kmax:
                # K_max dépassé : paquet abandonné
                self.dropped_packets += 1
                station.packet = None
                station.contention_window = self.config.wmin
                station.retries = 0
                self._schedule_next_arrival(station_id, current_time)
                continue

            # BEB :  $W \leftarrow \min(2W+1, W_{\max})$ 
            station.contention_window = min(2 * station.contention_window
+ 1, self.config.wmax)
            station.backoff =
self._sample_backoff(station.contention_window)
            self.contenders.add(station_id)

        self._schedule_slot_tick_if_needed(current_time)
        return

        # RTS réussi : une seule station
        station_id = next(iter(self.current_transmission))
        # Chronologie : RTS_end + SIFS + CTS + SIFS + DATA
        data_end_time = current_time + self.config.sifs +
self.config.cts_duration + self.config.sifs + self.config.packet_duration

        # NAV : autres stations bloquent leur backoff jusqu'à la fin de la
transmission
        for s in range(len(self.stations)):
            if s == station_id:
                continue
            st = self.stations[s]
            if st.packet is not None:
                st.nav_until = data_end_time

        self._push_event(data_end_time, EVENT_DATA_END, -1)

    def _handle_data_end(self, current_time: float) -> None:
        """Fin DATA : succès → enregistre métriques, libère station ;
collision → BEB ou abandon."""
        if self.current_transmission is None:
            return

        is_collision = len(self.current_transmission) > 1
        if not is_collision:
            # Transmission réussie
            station_id = next(iter(self.current_transmission))
            station = self.stations[station_id]
            self.successful_packets += 1
            self.successful_bits += self.config.packet_bits
            self.delay_sum += current_time - station.packet.arrival_time #
type: ignore[union-attr]

```

```

        station.packet = None
        station.contention_window = self.config.wmin
        station.retries = 0
        self.current_transmission = None
        self.contention_open_time = current_time + self.config.difs #
Prochaine contention après DIFS
        self._schedule_next_arrival(station_id, current_time)
        self._schedule_slot_tick_if_needed(current_time)
        return

# Collision DATA (rare avec RTS/CTS ; possible sans)
affected_stations = list(self.current_transmission)
self.collided_packets += len(affected_stations)
self.current_transmission = None
self.contention_open_time = current_time + self.config.difs

for station_id in affected_stations:
    station = self.stations[station_id]
    station.retries += 1
    if station.retries > self.config.kmax:
        # K_max dépassé : paquet abandonné
        self.dropped_packets += 1
        station.packet = None
        station.contention_window = self.config.wmin
        station.retries = 0
        self._schedule_next_arrival(station_id, current_time)
        continue

    # BEB :  $W \leftarrow \min(2W+1, W_{\max})$ 
    station.backoff = self._sample_backoff(station.contention_window)
    self.contenders.add(station_id)

self._schedule_slot_tick_if_needed(current_time)

def run_single_experiment(config: SimulationConfig) -> SimulationResult:
    """Crée et lance un CSMACASimulator ; retourne les métriques."""
    simulator = CSMACASimulator(config)
    return simulator.run()

def average_results(results: list[SimulationResult]) -> SimulationResult:
    """Moyenne des métriques continues, somme des compteurs entiers. Lève
    ValueError si vide."""
    if not results:
        raise ValueError("results must not be empty")

    return SimulationResult(
        throughput_packets_per_s=mean(result.throughput_packets_per_s for
result in results),
        throughput_bits_per_s=mean(result.throughput_bits_per_s for result in
results),
        channel_utilization=mean(result.channel_utilization for result in
results),
        offered_load_packets_per_s=mean(result.offered_load_packets_per_s for
result in results),
        collision_rate=mean(result.collision_rate for result in results),

```

```

        mean_delay_s=mean(result.mean_delay_s for result in results),
        generated_packets=sum(result.generated_packets for result in
results),
        successful_packets=sum(result.successful_packets for result in
results),
        dropped_packets=sum(result.dropped_packets for result in results),
        total_attempts=sum(result.total_attempts for result in results),
        collided_packets=sum(result.collided_packets for result in results),
    )

def sweep_stations(
    base_config: SimulationConfig,
    start: int,
    stop: int,
    step: int,
    runs: int,
) -> list[ExperimentPoint]:
    """Balaye N de start à stop par pas step, `runs` répétitions par point.

    Retourne une liste d'ExperimentPoint (moyenne  $\pm \sigma$ ). Lève ValueError si
step/runs <= 0.
    """
    if step <= 0:
        raise ValueError("step must be positive")
    if runs <= 0:
        raise ValueError("runs must be positive")

    points: list[ExperimentPoint] = []
    for station_count in range(start, stop + 1, step):
        run_results: list[SimulationResult] = []
        for repeat_index in range(runs):
            config = SimulationConfig(
                station_count=station_count,
                arrival_rate=base_config.arrival_rate,
                simulation_time=base_config.simulation_time,
                packet_bits=base_config.packet_bits,
                packet_duration=base_config.packet_duration,
                slot_time=base_config.slot_time,
                difs=base_config.difs,
                sifs=base_config.sifs,
                wmin=base_config.wmin,
                wmax=base_config.wmax,
                kmax=base_config.kmax,
                seed=None if base_config.seed is None else base_config.seed +
repeat_index + station_count * 1000,
                rtscts=base_config.rtscts,
                rts_duration=base_config.rts_duration,
                cts_duration=base_config.cts_duration,
            )
            run_results.append(run_single_experiment(config))

        averaged = average_results(run_results)
        throughput_bits_std = pstdev(r.throughput_bits_per_s for r in
run_results)
        collision_rate_std = pstdev(r.collision_rate for r in run_results)
        mean_delay_std = pstdev(r.mean_delay_s for r in run_results)

```

```

        points.append(
            ExperimentPoint(
                x_value=station_count,
                throughput_packets_per_s=averaged.throughput_packets_per_s,
                throughput_bits_per_s=averaged.throughput_bits_per_s,
                throughput_bits_std=throughput_bits_std,
                collision_rate=averaged.collision_rate,
                collision_rate_std=collision_rate_std,
                mean_delay_s=averaged.mean_delay_s,
                mean_delay_std=mean_delay_std,
            )
        )

    return points

def sweep_wmin(
    base_config: SimulationConfig,
    start: int,
    stop: int,
    step: int,
    runs: int,
) -> list[ExperimentPoint]:
    """Balaye W_min de start à stop par pas step, `runs` répétitions par
    point.

    W_max est ajusté à max(wmax, wmin) pour maintenir wmin <= wmax.
    Lève ValueError si step/runs <= 0.
    """
    if step <= 0:
        raise ValueError("step must be positive")
    if runs <= 0:
        raise ValueError("runs must be positive")

    points: list[ExperimentPoint] = []
    for wmin in range(start, stop + 1, step):
        run_results: list[SimulationResult] = []
        for repeat_index in range(runs):
            config = SimulationConfig(
                station_count=base_config.station_count,
                arrival_rate=base_config.arrival_rate,
                simulation_time=base_config.simulation_time,
                packet_bits=base_config.packet_bits,
                packet_duration=base_config.packet_duration,
                slot_time=base_config.slot_time,
                difs=base_config.difs,
                sifs=base_config.sifs,
                wmin=wmin,
                wmax=max(base_config.wmax, wmin),
                kmax=base_config.kmax,
                seed=None if base_config.seed is None else base_config.seed +
repeat_index + wmin * 1000,
                rtscts=base_config.rtscts,
                rts_duration=base_config.rts_duration,
                cts_duration=base_config.cts_duration,
            )
            run_results.append(run_single_experiment(config))

```



```

        averaged = average_results(run_results)
        throughput_bits_std = pstdev(r.throughput_bits_per_s for r in
run_results)
        collision_rate_std = pstdev(r.collision_rate for r in run_results)
        mean_delay_std = pstdev(r.mean_delay_s for r in run_results)
        points.append(
            ExperimentPoint(
                x_value=wmin,
                throughput_packets_per_s=averaged.throughput_packets_per_s,
                throughput_bits_per_s=averaged.throughput_bits_per_s,
                throughput_bits_std=throughput_bits_std,
                collision_rate=averaged.collision_rate,
                collision_rate_std=collision_rate_std,
                mean_delay_s=averaged.mean_delay_s,
                mean_delay_std=mean_delay_std,
            )
        )

    return points

```

```

def sweep_kmax(
    base_config: SimulationConfig,
    start: int,
    stop: int,
    step: int,
    runs: int,
) -> list[ExperimentPoint]:
    """Balaye K_max de start à stop par pas step, `runs` répétitions par
point.

```

K_max faible → abandons rapides ; K_max élevé → BEB converge, délai plus long.

Lève ValueError si step/runs <= 0.

```

"""
if step <= 0:
    raise ValueError("step must be positive")
if runs <= 0:
    raise ValueError("runs must be positive")

```

```

points: list[ExperimentPoint] = []
for kmax in range(start, stop + 1, step):
    run_results: list[SimulationResult] = []
    for repeat_index in range(runs):
        config = SimulationConfig(
            station_count=base_config.station_count,
            arrival_rate=base_config.arrival_rate,
            simulation_time=base_config.simulation_time,
            packet_bits=base_config.packet_bits,
            packet_duration=base_config.packet_duration,
            slot_time=base_config.slot_time,
            difs=base_config.difs,
            sifs=base_config.sifs,
            wmin=base_config.wmin,
            wmax=base_config.wmax,
            kmax=kmax,

```

```

        seed=None if base_config.seed is None else base_config.seed +
repeat_index + kmax * 1000,
        rtscts=base_config.rtscts,
        rts_duration=base_config.rts_duration,
        cts_duration=base_config.cts_duration,
    )
    run_results.append(run_single_experiment(config))

    averaged = average_results(run_results)
    throughput_bits_std = pstdev(r.throughput_bits_per_s for r in
run_results)
    collision_rate_std = pstdev(r.collision_rate for r in run_results)
    mean_delay_std = pstdev(r.mean_delay_s for r in run_results)
    points.append(
        ExperimentPoint(
            x_value=kmax,
            throughput_packets_per_s=averaged.throughput_packets_per_s,
            throughput_bits_per_s=averaged.throughput_bits_per_s,
            throughput_bits_std=throughput_bits_std,
            collision_rate=averaged.collision_rate,
            collision_rate_std=collision_rate_std,
            mean_delay_s=averaged.mean_delay_s,
            mean_delay_std=mean_delay_std,
        )
    )

    return points

def build_arg_parser() -> argparse.ArgumentParser:
    """Construit le parseur CLI (paramètres SimulationConfig + balayages
paramétriques)."""
    parser = argparse.ArgumentParser(description="Simulateur à événements
discrets du protocole CSMA/CA avec backoff exponentiel binaire")
    # --- Paramètres de la simulation ---
    parser.add_argument("--stations", type=int, default=8, help="Nombre de
stations en compétition pour le canal")
    parser.add_argument("--arrival-rate", type=float, default=20.0,
help="Taux d'arrivée par station (paquets/s) - processus de renouvellement,
non un Poisson pur")
    parser.add_argument("--simulation-time", type=float, default=20.0,
help="Horizon de simulation (secondes)")
    parser.add_argument("--packet-bits", type=int, default=12000,
help="Taille d'un paquet (bits) - 12 000 bits = 1 500 octets")
    parser.add_argument("--packet-duration", type=float, default=0.001,
help="Durée de transmission d'un paquet (secondes)")
    parser.add_argument("--slot-time", type=float, default=20e-6, help="Durée
d'un slot de backoff (secondes)")
    parser.add_argument("--difs", type=float, default=50e-6, help="DIFS :
délai inter-trames distribué (secondes)")
    parser.add_argument("--sifs", type=float, default=10e-6, help="SIFS :
délai inter-trames court (secondes)")
    # --- Paramètres du protocole CSMA/CA ---
    parser.add_argument("--wmin", type=int, default=15, help="Fenêtre de
contention minimale W_min")
    parser.add_argument("--wmax", type=int, default=1023, help="Fenêtre de
contention maximale W_max")

```

```

    parser.add_argument("--kmax", type=int, default=15, help="Nombre maximal
de tentatives avant abandon du paquet")
    # --- Contrôle de la simulation ---
    parser.add_argument("--seed", type=int, default=None, help="Graine
aléatoire pour la reproductibilité (None = non déterministe)")
    parser.add_argument("--runs", type=int, default=1, help="Nombre de
répétitions (résultats moyennés pour réduire la variance)")
    parser.add_argument("--output", type=Path,
default=Path("csma_ca_results.svg"), help="Chemin du fichier SVG généré")
    parser.add_argument("--csv", type=Path, default=None, help="Chemin
optionnel pour sauvegarder les résultats bruts en CSV")
    # --- Mode RTS/CTS ---
    parser.add_argument("--rtscts", action="store_true", help="Active le
mécanisme RTS/CTS avec réservation NAV")
    parser.add_argument("--rts-duration", type=float, default=200e-6,
help="Durée d'une trame RTS (secondes)")
    parser.add_argument("--cts-duration", type=float, default=200e-6,
help="Durée d'une trame CTS (secondes)")
    parser.add_argument("--title", type=str, default=None, help="Titre
personnalisé du graphique SVG (remplace le titre généré automatiquement)")

    # --- Balayages paramétriques (mutuellement exclusifs) ---
    sweep_group = parser.add_mutually_exclusive_group()
    sweep_group.add_argument("--sweep-stations", nargs=3, type=int,
metavar=("START", "STOP", "STEP"), help="Balayage du nombre de stations
(début fin pas)")
    sweep_group.add_argument("--sweep-wmin", nargs=3, type=int,
metavar=("START", "STOP", "STEP"), help="Balayage de la fenêtre de contention
minimale W_min (début fin pas)")
    sweep_group.add_argument("--sweep-kmax", nargs=3, type=int,
metavar=("START", "STOP", "STEP"), help="Balayage du nombre maximal de
tentatives K_max (début fin pas)")

    return parser

def main() -> None:
    """Point d'entrée CLI : parse les arguments, lance la simulation ou le
balayage, affiche et sauvegarde."""
    args = build_arg_parser().parse_args()
    config = SimulationConfig(
        station_count=args.stations,
        arrival_rate=args.arrival_rate,
        simulation_time=args.simulation_time,
        packet_bits=args.packet_bits,
        packet_duration=args.packet_duration,
        slot_time=args.slot_time,
        difs=args.difs,
        sifs=args.sifs,
        wmin=args.wmin,
        wmax=args.wmax,
        kmax=args.kmax,
        seed=args.seed,
        rtscts=args.rtscts,
        rts_duration=args.rts_duration,
        cts_duration=args.cts_duration,
    )

```

```

if args.runs <= 0:
    raise SystemExit("--runs must be positive")

if args.sweep_stations is not None:
    start, stop, step = args.sweep_stations
    points = sweep_stations(config, start, stop, step, args.runs)
    print("Station sweep")
    for point in points:
        print(
            f"N={point.x_value:3d} |
throughput={point.throughput_bits_per_s:12.2f} bits/s | "
            f"collision={point.collision_rate * 100:6.2f} % |
delay={point.mean_delay_s * 1000:8.4f} ms"
        )
        plot_points(points, args.title, "Nombre de stations", args.output)
        print(f"Plot saved to {args.output}")
    if args.csv is not None:
        save_csv(points, None, None, args.csv)
        print(f"CSV saved to {args.csv}")
    return

if args.sweep_wmin is not None:
    start, stop, step = args.sweep_wmin
    points = sweep_wmin(config, start, stop, step, args.runs)
    print("Wmin sweep")
    for point in points:
        print(
            f"Wmin={point.x_value:3d} |
throughput={point.throughput_bits_per_s:12.2f} bits/s | "
            f"collision={point.collision_rate * 100:6.2f} % |
delay={point.mean_delay_s * 1000:8.4f} ms"
        )
        plot_points(points, args.title, "Fenêtre de contention minimale
(Wmin)", args.output)
        print(f"Plot saved to {args.output}")
    if args.csv is not None:
        save_csv(points, None, None, args.csv)
        print(f"CSV saved to {args.csv}")
    return

if args.sweep_kmax is not None:
    start, stop, step = args.sweep_kmax
    points = sweep_kmax(config, start, stop, step, args.runs)
    print("Kmax sweep")
    for point in points:
        print(
            f"Kmax={point.x_value:3d} |
throughput={point.throughput_bits_per_s:12.2f} bits/s | "
            f"collision={point.collision_rate * 100:6.2f} % |
delay={point.mean_delay_s * 1000:8.4f} ms"
        )
        plot_points(points, args.title, "Nombre maximal de tentatives K_max",
args.output)
        print(f"Plot saved to {args.output}")
    if args.csv is not None:
        save_csv(points, None, None, args.csv)

```

```

        print(f"CSV saved to {args.csv}")
    return

if args.runs == 1:
    result = run_single_experiment(config)
else:
    run_results = [
        run_single_experiment(
            SimulationConfig(
                station_count=config.station_count,
                arrival_rate=config.arrival_rate,
                simulation_time=config.simulation_time,
                packet_bits=config.packet_bits,
                packet_duration=config.packet_duration,
                slot_time=config.slot_time,
                difs=config.difs,
                sifs=config.sifs,
                wmin=config.wmin,
                wmax=config.wmax,
                kmax=config.kmax,
                seed=None if config.seed is None else config.seed +
index,

                rtscts=config.rtscts,
                rts_duration=config.rts_duration,
                cts_duration=config.cts_duration,
            )
        )
        for index in range(args.runs)
    ]
    result = average_results(run_results)

print_result(config, result)
if args.csv is not None:
    save_csv(None, result, config, args.csv)
    print(f"CSV saved to {args.csv}")

if __name__ == "__main__": # pragma: no cover
    main() # pragma: no cover

```